

進階程式設計

Advanced Program Design

L7 Part I



Chia-Chi Chang

ccchang@cc.ncue.edu.tw

因數(Factor)

※a和b存在整除的關係，若b可以被a整除，則a是b的因數，b是a的倍數。

➤ 給定1個整數，找尋所有因數

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int main(int argc, char *argv[]) {
    int16_t m, loop;

    scanf("%hd", &m);
    for(loop=1; loop<=m; loop++) {
        if(!(m % loop)) {
            printf("%hd ", loop);
        }
    }
    return 0;
}
```

窮舉法
時間複雜度 $O(n)$

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int main(int argc, char *argv[]) {
    int16_t m, loop;

    scanf("%hd", &m);
    for(loop=1; loop*loop<m; loop++) {
        if(!(m % loop)) {
            printf("%hd ", loop);
            printf("%hd ", m / loop);
        }
    }
    if((loop * loop) == m) {
        printf("%hd ", loop);
    }
    return 0;
}
```

$m = \text{loop} * q$

窮舉法改善
時間複雜度 $O(\sqrt{n})$

最大公因數(Greatest Common Divisor, GCD)

※若兩數存在最大公因數，則兩數各為最大公因數的整數倍

窮舉法(暴力法)

1. 取兩數a,b之中的最小數
2. 若b為最小數，嘗試1~b所有數字是否能同時整除a和b，取其最大者。

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int16_t gcd(int16_t, int16_t);
int main(int argc, char *argv[]) {
    int16_t m, n;

    scanf("%hd %hd", &m, &n);
    printf("gcd = %hd\n", gcd(m, n));
    return 0;
}

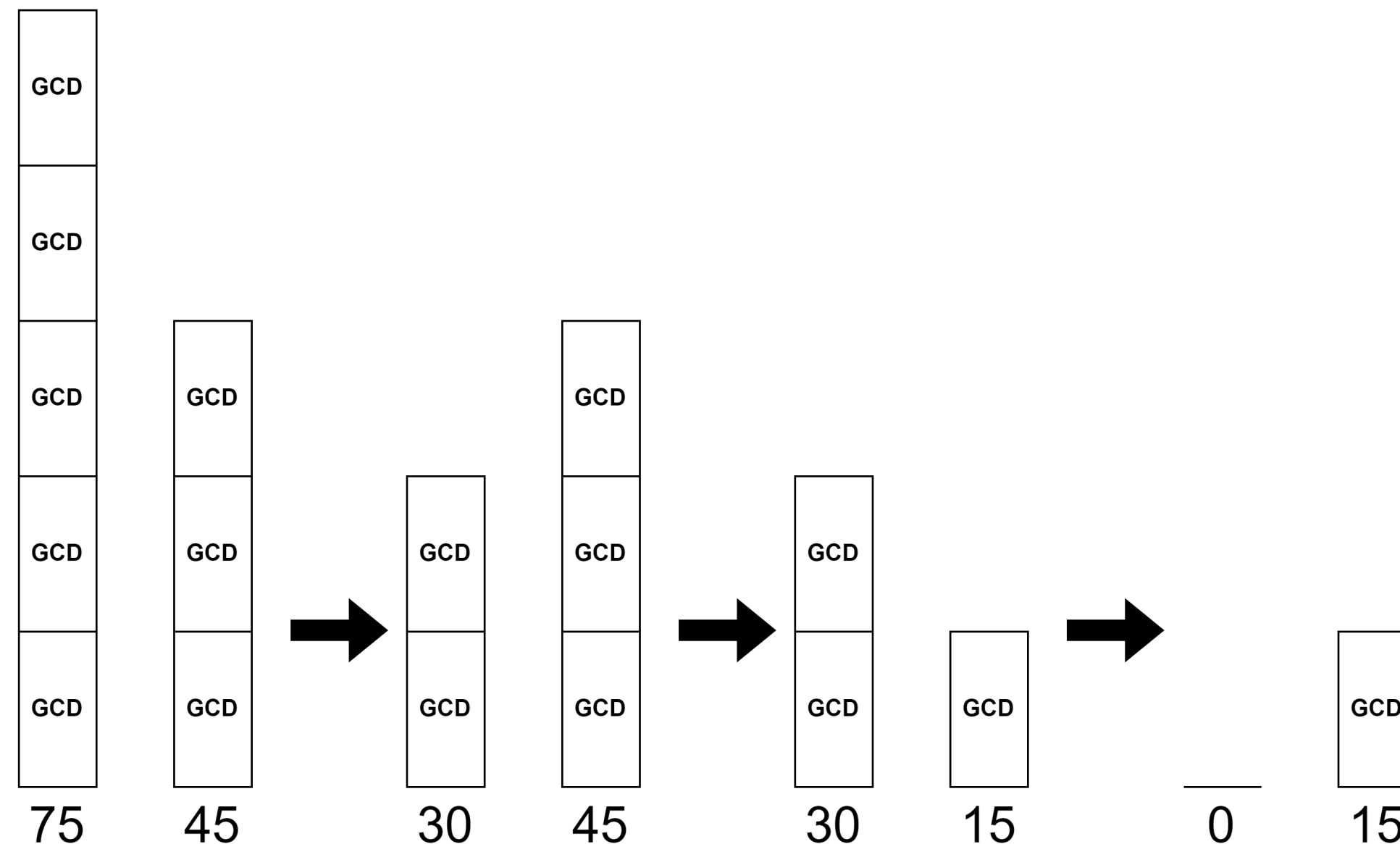
int16_t gcd(int16_t a, int16_t b) {
    int16_t loop, loopMax, v;

    if(a < b) {
        loopMax = a;
    } else {
        loopMax = b;
    }
    for(loop=1; loop<=a; loop++) {
        if(((a % loop) == 0) && ((b % loop) == 0)) {
            v = loop;
        }
    }
    return v;
}
```

最大公因數(Greatest Common Divisor, GCD)

※歐幾里德演算法(輾轉相除法)

➤ 兩數輾轉相除之餘數也會是最大公因數的整數倍



Euclidean Algorithm
輾轉相除法

最大公因數(Greatest Common Divisor, GCD)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int16_t gcd(int16_t, int16_t);
int main(int argc, char *argv[]) {
    int16_t m, n;

    scanf("%hd %hd", &m, &n);
    printf("gcd = %hd\n", gcd(m, n));
    return 0;
}

int16_t gcd(int16_t a, int16_t b) {
    int16_t t;

    while(b != 0) {
        t = a % b;
        a = b;
        b = t;
    }
    return a;
}
```

Euclidean Algorithm(迴圈版本)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int16_t gcd(int16_t, int16_t);
int main(int argc, char *argv[]) {
    int16_t m, n;

    scanf("%hd %hd", &m, &n);
    printf("gcd = %hd\n", gcd(m, n));
    return 0;
}

int16_t gcd(int16_t a, int16_t b) {
    if(b == 0) return a;
    return gcd(b, a % b);
}
```

Euclidean Algorithm(遞迴版本)

最小公倍數(Least Common Multiple, LCM)

※利用GCD求a和b的最小公倍數

- $LCM(a, b) = \frac{|ab|}{GCD(a, b)} = |a| \frac{|b|}{GCD(a, b)} = |b| \frac{|a|}{GCD(a, b)}$
- $LCM(21, 6) = \frac{|21 \times 6|}{GCD(21, 6)} = |21| \frac{|6|}{GCD(21, 6)} = 21 \times \frac{6}{3} = 42$

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int16_t gcd(int16_t, int16_t);
int32_t lcm(int16_t, int16_t);
int main(int argc, char *argv[]) {
    int16_t m, n;

    scanf("%hd %hd", &m, &n);
    printf("gcd = %hd\n", gcd(m, n));
    printf("lcm = %d\n", lcm(m, n));
    return 0;
}

int16_t gcd(int16_t a, int16_t b) {
    if(b == 0) return a;
    return gcd(b, a % b);
}

int32_t lcm(int16_t a, int16_t b) {
    return a * b / gcd(a, b);
}
```

質數(Prime Number)

- 正整數分成1、質數(Prime Number)、合數(Composite Number)
- 質數：除了1和自己本身之外，無法被其他數整除的數稱為質數
- 合數：除了1和自己本身之外，還有其他因數。

※除了1之外，若該數不是合數的話，就代表是質數。

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int16_t checkIsPrime(int16_t p) {
    int16_t loop;

    for(loop=2; loop<p; loop++) {
        if(!(p % loop)) {
            return 0;
        }
    }
    return 1;
}

int main(int argc, char *argv[]) {
    int16_t p;

    while(scanf("%hd", &p) != EOF) {
        if(checkIsPrime(p)) {
            printf("%d is a prime number!\n",p);
        } else {
            printf("%d is not a prime number!\n",p);
        }
    }
    return 0;
}
```

檢查所有數字是否為質數(窮舉法)

質數(Prime Number)

邏輯證明：

1. 假設 n 是合數，則必有 $n = a * b$ ，且 $1 < a \leq b < n$ 。
2. 如果 $a > \sqrt{n}$ ，且 $b > \sqrt{n}$ ，則導致 $a * b > \sqrt{n} * \sqrt{n} = n$ 。
這與假設 $n = a * b$ 矛盾。
3. 因此，若 n 為合數，必然存在一個因子 a 滿足 $1 < a < \sqrt{n}$ 。

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
```

使用乘法比起sqrt(p)來的有效率

$\text{loop} \leq \sqrt{p}$

```
int16_t checkIsPrime(int16_t p) {
    int16_t loop;
    for(loop=2; loop*loop <= p; loop++) {
        if(!(p % loop)) {
            return 0;
        }
    }
    return 1;
}
```

$= (\text{loop})^2 \leq (\sqrt{p})^2$

$= \text{loop} \times \text{loop} \leq p$

```
int main(int argc, char *argv[]) {
    int16_t p;

    while(scanf("%hd", &p) != EOF) {
        if(checkIsPrime(p)) {
            printf("%d is a prime number!\n", p);
        } else {
            printf("%d is not a prime number!\n", p);
        }
    }
    return 0;
}
```

只需檢查至sqrt(p)即可

判斷p是否為質數的策略

※先拿什麼數去除p，可以讓程式進一步優化

1) 判斷是否為2或3的倍數，這樣可以節省至少一半的時間

2) 觀察剩餘數的變化: 5, 7, 11, 13, 17, 19, 23, 25, 29, 31,...

➤ int32_t isPrimeNumber(int32_t p)用來檢查p是否為質數；回傳0代表不是質數，回傳1則代表質數

```
int32_t isPrimeNumber(int32_t p) {  
    if(p <= 1) return 0;  
    if(p == 2) return 1;  
    if(p == 3) return 1;  
  
    if(!(p % 2)) return 0;  
    if(!(p % 3)) return 0;  
  
    for(int32_t loop=5, gap=2; loop*loop<=p; loop+=gap, gap=6-gap) {  
        if(!(p % loop)) return 0;  
    }  
    return 1;  
}
```

C參考範例

The End

